User Guide: Compass-Needle Lattice Simulator

J. P. Sinnecker

Generated 2026-10-18 from USER_GUIDE.md (commit 20c3bad)

Contents

1 User Guide: Compass-Needle Lattice Simulator (`compass_sim`) 1
- 1.1 Table of Contents . . . 2
- 1.2 1. Physics & Theoretical Model . . . 2
- 1.3 2. Lattice Geometry Options . . . 4
- 1.4 3. External Field Modes (`--field_mode`) . . . 4
- 1.5 4. Demagnetization Modes . . . 4
- 1.6 4b. Event-Driven Quasi-Static Loop (`--field_mode hysteresis_qs`) . . . 5
- 1.7 5. System Requirements & Acceleration . . . 6
- 1.8 6. Command Line Parameters Reference . . . 6
- 1.9 7. Output Files & Data Formats . . . 9
- 1.10 8. Step-by-Step Practical Examples . . . 9
- 1.11 9. Running Sweep Campaigns . . . 10
- 1.12 10. Summary Reference Table of Parameters . . . 11
- 1.13 11. Validation & Quality Control . . . 12

1 User Guide: Compass-Needle Lattice Simulator (`compass_sim`)

Compiled Version: revised 2026-10-16 for `compass.py` 2.1.0 plus the Phase-1 physics fixes (branches `fase1/physics-fixes` and `fase1/sweep-rate-dimless`). Flag names, defaults and choice lists were checked against `build_parser()`. The first rewrite (2026-07-13) replaced a guide that described a dead lineage of the simulator (see `docs/AUDIT.md`, P1 item 6). For the physics audit and the publication roadmap, see `docs/AUDIT_PHYSICS_ROADMAP.md`.

Welcome to the **Compass-Needle Lattice Simulator** (`compass_sim`), a Python-based physics simulation engine that models the dynamic behavior of 2D arrays of interacting classical magnetic dipoles (compass needles).

The simulation models real Newtonian rotational dynamics in SI units. It integrates the equation of motion for each needle with a damped **velocity-Verlet** scheme: the damping is treated implicitly, and the scheme is symplectic at $b = 0$. It runs on CPU (NumPy) by default and can optionally use a GPU (CuPy) backend via `--use_gpu`.

> **What this model is and is not.** It describes a macroscopic array of inertial dipolar rotors, the tabletop compass-needle system. It is **not** a model of nanomagnetic artificial spin ice: those islands follow gyromagnetic (LLG) dynamics and have shape-anisotropy switching barriers. Artificial spin ice is motivation only. Do not claim avalanche exponents or "universality classes" from the array sizes accessible here.

1.0.1 Breaking changes of October 2026 (Phase 1)

Runs made after these changes **cannot be compared number by number** with older runs, including the older figures in `figs/`:

What	Before	Now
Dipolar sum	cutoff 3.5 r_{nn} (interior field ~15–17 % too low, no edge fields)	full pair sum (`--cutoff_shells 0`)
Time step	`dt_factor`·T_0 with T_0 from B_{ref} ($\omega_{\text{loc}}\Delta t \approx 0.5$)	`dt_factor`·T_{loc} from a worst-case local-field bou
`Q` in metadata	$\omega_0 I/b$ with $B_{\text{eff}}=\max(B_{\text{ref}},$	B_{ext}
Energy	$E_{\text{dip}}, E_{\text{ext}}, E_{\text{kin}}$ only	plus the ledger `W_field_J`, `E_diss_J` and `energ`

What	Before	Now
Sweep rate	implicit in `--t_sim`	explicit dimensionless r (`--sweep_rate_dimle`

Each change is covered by a test in `tests/test_physics.py` (see §11).

1.1 Table of Contents

1. Physics & Theoretical Model
2. Lattice Geometry Options
3. External Field Modes (`--field_mode`)
4. Demagnetization Modes 4b. Event-Driven Quasi-Static Loop
5. System Requirements & Acceleration
6. Command Line Parameters Reference
7. Output Files & Data Formats
8. Step-by-Step Practical Examples
9. Running Sweep Campaigns
10. Summary Reference Table of Parameters
11. Validation & Quality Control

1.2 1. Physics & Theoretical Model

At the core of the simulator is the rotational equation of motion for each needle i:

$$I\ddot{\theta}_i = \tau_i^{\text{dip}} + \tau_i^{\text{ext}} - b_i\dot{\theta}_i$$

Where:

- θ_i: Orientation angle of the i-th compass needle in the 2D plane (xy-plane).
- I: Moment of inertia of each needle ($kg \cdot m^2$).
- $\tau_i^{\mathbf{dip}}$: Rotational torque due to dipole-dipole interactions with all other needles (by default the full pair sum over the finite array; an optional cutoff is set with `--cutoff_shells`/`--cutoff_m`).
- $\tau_i^{\mathbf{ext}}$: Rotational torque due to the applied external magnetic field ($B_{\text{ext}}(t)$).
- b_i: Viscous rotational damping coefficient, optionally randomized per needle via `--damping_noise` ($b_i = b\,[1 + \eta_i]$, $\eta_i \in [-\delta_b, \delta_b]$).

There is no stochastic thermal-torque term in the dynamics: the model is purely deterministic Newtonian rotation plus viscous damping, i.e. $T = 0$. Randomness only enters through the initial angular noise (`--noise`) and the optional per-needle damping variation above.

Known model limitations (see `docs/AUDIT_PHYSICS_ROADMAP.md`, §2.6):

- **Point dipoles at the pivots.** The default needle length (10 mm) is 0.77 of the pivot spacing (13 mm), so near-field (multipolar) corrections are not small. A dumbbell (end-charge) model is planned as a robustness check.
- **No dry (Coulomb) pivot friction.** The real apparatus has static friction, which acts as a switching threshold. The model has only viscous damping, so it has no intrinsic switching-field distribution.
- **No thermal bath.** Results are deterministic for a given seed.

1.2.1 Automatic Property Calibration

If `--moment`/`--inertia` are not supplied explicitly, they are derived from a **rhombus-shaped** blade geometry (not a rectangular sheet):

- **Blade area**: $A = 0.5 \cdot L \cdot W$, where $L =$ `--needle_len` and $W =$ `--needle_width` (independent of lattice spacing by default; see `--use_legacy_size_from_R` below for the legacy convention).

- **Moment of inertia**: $I = \frac{1}{24} m_{\text{blade}}(L^2 + W^2)$ for the solid blade about its center, with an optional pivot-hole subtraction and pivot-cylinder addition (`--pivot_radius`, `--pivot_thickness`, `--pivot_density`, or an explicit `--pivot_mass` override).
- **Magnetic moment**: $m = M_s \cdot A_{\text{net}} \cdot t$, where $t =$ `--needle_thickness`, A_{net} subtracts the pivot-hole area if any, and $M_s =$ `--steel_Ms` (or B_{sat}/μ_0 if `--steel_Bsat` is given instead).

Two needle-thickness defaults coexist: `compass.py` uses `--needle_thickness 0.0004` (0.4 mm), while `damping_sweepV03.py` (via `config.yaml`, section `damping_sweep`) and `calc_defaults_temp.py` use 0.26 mm, as do the project report and the needle drawing. This has not yet been reconciled against the real needle. Always pass `--needle_thickness` explicitly in production runs, and check the value recorded in the metadata.

1.2.2 Natural frequency and quality factor

The reference field and frequency are built from the nearest-neighbor dipolar field. They are **normalization units**, not the actual local field, which is $\approx 2\,B_{\text{ref}}$ in an ordered array:

$$B_{\text{ref}} = \frac{\mu_0}{4\pi}\frac{2m}{r_{nn}^3}, \qquad \omega_0 = \sqrt{\frac{m\,B_{\text{eff}}}{I}}, \qquad B_{\text{eff}} = \max(B_{\text{ref}}, |B_{\text{ext}}|)$$

$T_0 = 2\pi/\omega_0$ is the time unit for dwell times and sweep rates. The integration timestep is $\Delta t = \text{dt_factor} \cdot T_{\text{loc}}$, where T_{loc} is the worst-case local period (see `--dt_factor`).

The quality factor reported as `Q` in the metadata is the **intrinsic** Q_0. It depends on the array and the damping only, not on the field protocol:

$$Q_0 = \frac{\omega_{\text{lat}} I}{b}, \qquad \omega_{\text{lat}} = \sqrt{\frac{m\,B_{\text{lat}}}{I}},$$

where B_{lat} is the lattice field scale: the median over sites of $|B_i^{\text{dip}}|$ in uniform states, averaged over 8 in-plane directions ($\approx 2.2\,B_{\text{ref}}$ for large square arrays, smaller for small arrays because of their edges). Q_0 characterizes the damping regime ($Q_0 \ll 1$: overdamped; $Q_0 \gg 1$: underdamped/ringing). The metadata also records two other conventions:

- `Q_ref` $= \sqrt{mB_{\text{ref}}I}/b$, the convention used to build the damping grid in `damping_sweepV03.py`;
- `Q_field` $= \omega_0 I/b$, the old protocol-dependent definition (before October 2026, this was the value written as `Q`). It changes with `--B_max_factor` and is kept only for comparison with older data.

1.2.3 Dimensionless sweep rate r

Loop area, coercive field and avalanche statistics depend strongly on how fast the field is swept. A claim about the effect of Q_0 must therefore be made at the quasi-static limit ($r \to 0$) or as an explicit function of the dimensionless sweep rate

$$r = \frac{dB/dt \;\cdot\; \tau_{\text{relax}}}{B_{\text{ref}}}, \qquad \tau_{\text{relax}} = \max\!\left(\frac{1}{\omega_{\text{lat}}},\; \frac{b}{m\,B_{\text{lat}}}\right) = \frac{\max(1,\, 1/Q_0)}{\omega_{\text{lat}}}.$$

τ_{relax} is the needle relaxation time: the inertial response time $1/\omega_{\text{lat}}$ when underdamped ($Q_0 \geq 1$) and the viscous time $b/(mB_{\text{lat}})$ when overdamped ($Q_0 < 1$). b is the mean damping (`--damping_noise` is ignored). r is the field change, in units of B_{ref}, during one relaxation time: $r \ll 1$ means the array follows the field adiabatically.

- `--sweep_rate_dimless r` sets the rate directly. In `hysteresis` mode it fixes the base rate of the five-branch cycle, so $t_{\text{sim}} = 5\,|B_{\text{max}}|\,\tau_{\text{relax}}/(rB_{\text{ref}})$; it cannot be combined with `--t_sim`. In `forc` mode it fixes the ramp rate of every curve (the fixed ramp times are ignored); it cannot be combined with `--forc_rate`. Other modes reject it.
- Every `hysteresis` and `forc` run records the r it actually used in the metadata block `sweep_rate`, also when the ramp came from `--t_sim`, `--forc_rate` or fixed FORC ramp times. With fixed FORC ramp times each curve has its own rate; `sweep_rate_dimless` is then the fastest curve, and `forc_sweep_rate_dimless_per_curve` lists all of them. With `--hyst_spacing log` the rate is not constant along a branch and r is the branch-mean rate; inside a `--hyst_slow_window` the local rate is r/`hyst_slow_factor`.

- Cost: the number of steps grows as $1/r$. For a 6×6 triangular array at $Q_0 = 2$, $|B_{\max}| = 3B_{\text{ref}}$, the loop area was 1.03×10^{-3}, 2.80×10^{-4}, 1.62×10^{-4} and 1.46×10^{-4} J at $r = 10^{-1}, 10^{-2}, 10^{-3}, 3 \times 10^{-4}$: it converges, but only below $r \approx 10^{-3}$. Check convergence in r for each geometry and size before quoting a quasi-static value (`tests/test_physics.py::test_loop_area_converges_in_the_quasi_static_limit`).

1.3 2. Lattice Geometry Options

The layout of the needles affects the dipolar interaction field profile. The simulator supports three distinct 2D geometry layouts, selected using `--geometry`:

1.3.1 1. Square Lattice (`--geometry square`)

- A rectangular $N \times M$ grid of pivots.
- Four nearest neighbors in the bulk; the least frustrated of the three geometries.

1.3.2 2. Triangular Lattice (`--geometry triangular`)

- An equilateral triangular grid.
- Six nearest neighbors per bulk pivot — the highest local coordination of the three, introducing strong geometric frustration.

1.3.3 3. Honeycomb Lattice (`--geometry honeycomb`)

- A hexagonal grid with hexagonal voids (graphene-like), following a finite hexagonal-hole construction.
- Three nearest neighbors per bulk pivot — the lowest coordination of the three.

In all three cases the nearest-neighbor distance r_{nn} is computed directly from the generated site positions (not assumed to be exactly $2R$), though in the default construction $r_{nn} \simeq 2R$ for all geometries, where $R =$ `--R` is half the pivot-to-pivot spacing.

1.4 3. External Field Modes (`--field_mode`)

You can control how the external magnetic field $B_{\text{ext}}(t)$ changes over time using `--field_mode`. The full choice set is: `static`, `hysteresis`, `hysteresis_qs`, `forc`, `sine`, `step_up`/`step_pos`, `step_down`/`step_neg`, `pulse`, `demag_rot`, `demag_linear`.

Mode	Description
`static`	Applies a constant field B_{ext} in a fixed direction for the whole run.
`hysteresis`	Sweeps the field through a full five-segment cycle: $0 \to +B_{\max} \to 0 \to -B_{\max} \to 0 \to +B_{\max}$, each segm
`hysteresis_qs`	**Event-driven quasi-static** five-branch loop: the field advances in steps of `--qs_dB_factor` $\times B_{\text{ref}}$ and
`forc`	First-Order Reversal Curves: repeated cycles of hold-at-saturation, ramp down to a reversal field B_r (va
`sine`	A sinusoidal external field: $B_{\text{ext}}(t) = B_{\max} \sin(2\pi f t)$.
`step_up` / `step_pos`	Field is 0 until `--field_delay`, then steps to $+B_{\max}$ and stays there.
`step_down` / `step_neg`	Field is $B_{\max}$ until `--field_delay`, then steps to 0.
`pulse`	Field is 0 until `--field_delay`, then $B_{\max}$ for `--t_pulse` seconds, then 0 again (stays on indefinitely if
`demag_rot`	Rotating field whose magnitude decays linearly to zero over `--demag_cycles`/`--demag_freq` seconds,
`demag_linear`	Field magnitude decays linearly to zero along the fixed `--phi_ext_deg` direction (no rotation), over th

$B_{\max}$ is `--B_ext` if given, else `--B_max_factor` $\times B_{\text{ref}}$ (see §1).

1.5 4. Demagnetization Modes

Demagnetization is now expressed as a **field mode** (`--field_mode demag_rot` or `--field_mode demag_linear`), not a separate `--demag` flag layered on top of another mode. There is no simulated-annealing/thermal-noise demagnetization option in the current engine (that concept belonged to an earlier, now-archived lineage).

1.5.1 `demag_rot`

The external field vector rotates continuously in the xy-plane while its magnitude decays linearly to zero:

$$B_x(t) = B_{\max}\left(1 - \frac{t}{T_{\text{demag}}}\right)\cos(2\pi f_{\text{demag}} t), \qquad B_y(t) = B_{\max}\left(1 - \frac{t}{T_{\text{demag}}}\right)\sin(2\pi f_{\text{demag}} t)$$

where $T_{\text{demag}} = $ `--demag_cycles` / `--demag_freq`. Rotational demagnetization sweeps the field vector through the plane, forcing all needles through a complete angular cycle at decreasing amplitude — useful for erasing directional bias before a measurement.

1.5.2 `demag_linear`

The field decays linearly to zero along the fixed direction set by `--phi_ext_deg`, without rotating. Simpler than `demag_rot`, but does not sweep through all in-plane directions.

After either mode completes, `--t_relax_after` extra seconds of zero-field relaxation are appended automatically (this is folded into the mode's own total simulated time; `--t_sim` is ignored for these two modes, exactly as for `forc`).

1.6 4b. Event-Driven Quasi-Static Loop (`--field_mode hysteresis_qs`)

A continuous sweep only approaches the quasi-static loop as $r \to 0$, and for large arrays this limit was not reached even at $r = 10^{-3}$ (see `docs/FASE2_RESULTADOS_PRELIMINARES.md`). `hysteresis_qs` computes that limit directly:

1. the field follows the same five branches as `hysteresis` ($0 \to +B_{\max} \to 0 \to -B_{\max} \to 0 \to +B_{\max}$), in steps of $\Delta B =$ `--qs_dB_factor` $\times B_{\text{ref}}$;
2. after each step the needles evolve **at fixed field** with the real dynamics until rest: $\max_i |\tau_i| \le$ `--qs_tol_torque` $\times m B_{\text{lat}}$ **and** $\max_i |\omega_i| \le$ `--qs_tol_omega` $\times \omega_{\text{lat}}$ (checked every `--qs_check_every` steps);
3. **saddle check:** each needle gets a random kick of `--qs_kick` rad (default 10^{-4}, reproducible from `--seed`; areas are identical for 10^{-4}–10^{-2}, and smaller kicks relax faster), and the array relaxes again. A true minimum returns to the same state, while a saddle point (zero torque, but unstable) is abandoned;
4. the rest state is written as **one CSV row per field step**, and the next step follows.

Consequences:

- In a row, the `flip_field`/`flip_angle` counts and the increase of `E_diss_J` are the **avalanche size and energy dissipated by that field step**, which gives event-resolved statistics directly.
- The field jump enters the energy ledger exactly ($-\sum_i \mathbf{m}_i \cdot \Delta\mathbf{B}$ at fixed angles), and the metadata block `quasi_static` records the step, tolerances, the number of field steps and how many failed to converge (`unconverged_field_steps`, which should be 0).
- `--qs_dynamics inertial` (default) uses the real Q_0, so inertia can still decide where each avalanche ends (overshoot into another basin). `--qs_dynamics overdamped` integrates the gradient flow $b\dot{\theta} = \tau$ instead, which is the $Q_0 \to 0$ limit without inertia. In that case `--damping` only sets the time unit.
- `--sweep_rate_dimless` and `--t_sim` do not apply. `t_s` is the accumulated relaxation time.
- **Why the kick is needed:** a zero-torque state can be a saddle. The gradient flow preserves lattice symmetries exactly, so it can stay on a symmetric branch after that branch has become unstable. Without the check, small honeycomb arrays spent ~500 of 3200 field steps on saddles, and the 'inertial effect' of the first quasi-static grid was largely this artefact (`docs/QS_SELECAO_DE_CAMINHO.md`). The kick energy is booked as `E_kick_J` in `energy_ledger`. `--qs_kick 0` disables the check; use it only to reproduce the first grid.

Validated defaults (`--qs_dB_factor 0.005`, `--qs_tol_torque 1e-5`, `--qs_tol_omega 1e-5`). A loose tolerance (1e-3) stops early near instabilities, where the dynamics slows down critically, and **inflates the loop by up to 50 %**. With the defaults, the overdamped and $Q_0 = 0.3$ inertial loops agree to 0.1 %, and both match the slowest rate sweeps. The cost is comparable to a slow sweep (≈ 1.5–5×10^6 steps per loop), because the critical slowing down near instabilities must be resolved either way.

```
python3 compass.py --geometry square --N 16 --M 16 --field_mode hysteresis_qs \
    --B_max_factor 4 --damping 1e-7 --out_dir ~/compass_runs/qs_demo --tag sq16_qs
```

1.7 5. System Requirements & Acceleration

`compass_sim` is designed to scale from small testing runs to large arrays containing thousands of needles.

1.7.1 CPU Execution (Default)

Uses **NumPy** for vectorization. The dipolar field is computed from a precomputed $K \times K$ interaction tensor (three dense matrices), so each integration step is a matrix-vector product: $O(K^2)$ arithmetic, with no geometry to rebuild. Memory is $3K^2 \times 8$ bytes in double precision: about 25 MB for $K \approx 10^3$ (32×32) and about 2.4 GB for $K = 10^4$ (100×100). `--tensor_mem_limit_gb` guards it.

Since October 2026 the default time step is smaller ($\approx$ 2–4× more steps than before, see §1), and quasi-static hysteresis needs $r \lesssim 10^{-3}$, which multiplies the run time as $1/r$. For campaigns, prefer many independent CPU processes (`damping_sweepV03.py --n_workers`) over a single GPU context.

1.7.2 GPU Execution (`--use_gpu`)

Uses **CuPy** to run the same tensor operations on an NVIDIA GPU.

- **Requirements**: An NVIDIA GPU with CUDA drivers installed, and CuPy (`pip install cupy-cuda12x`, matching your CUDA toolkit version).
- **When it helps**: Large lattices where the $K \times K$ dipolar tensor dominates cost. `--float32` can further reduce memory pressure for exploratory GPU runs (double precision remains preferred for final results).

1.8 6. Command Line Parameters Reference

This table mirrors `compass.py`'s `build_parser()` argument groups directly. Some flags have no `help=` string in the source (noted below); their descriptions here are otherwise hand-authored and could in principle drift — cross-check with `python3 compass.py --help` or `tools/generate_cli_reference.py` if in doubt.

1.8.1 Lattice geometry

- `--geometry {square,triangular,honeycomb}`: Lattice topology (default: `square`).
- `--N <int>`: Rows, or nominal honeycomb height (default: `16`).
- `--M <int>`: Columns, or nominal honeycomb width (default: `16`).
- `--R <float>`: Half the pivot-to-pivot spacing in meters; default gives 13 mm spacing (default: `0.0065`).
- `--needle_frac <float>`: Legacy blade-length fraction of $2R$, used only with `--use_legacy_size_from_R` (default: `0.80`).
- `--needle_len <float>`: Physical needle blade length in meters, independent of lattice spacing (default: `0.010`).
- `--needle_width <float>`: Physical needle blade width in meters (default: `0.003`).
- `--use_legacy_size_from_R {0,1}`: If `1`, derive blade size from `needle_frac*2R` instead of the explicit `needle_len`/`needle_width` above (default: `0`).

1.8.2 Needle physical properties

- `--moment <float>`: Override magnetic moment m in A·m² (default: `None`, auto-derived).
- `--inertia <float>`: Override moment of inertia I in kg·m² (default: `None`, auto-derived).
- `--needle_thickness <float>`: Blade thickness in meters (default: `0.0004`).
- `--steel_density <float>`: Blade material density in kg/m³ (default: `7850.0`).
- `--steel_Ms <float>`: Saturation magnetization in A/m (default: `1.59e6`, $\approx$ 2.0 T saturation flux density).
- `--steel_Bsat <float>`: Saturation flux density in Tesla; if given, overrides `--steel_Ms` via $M_s = B_{\text{sat}}/\mu_0$ (default: `None`).
- `--pivot_radius <float>`: Pivot hole/cylinder radius in meters (default: `0.0`, no pivot correction).
- `--pivot_thickness <float>`: Pivot cylinder height in meters (default: `0.0`).
- `--pivot_density <float>`: Pivot material density in kg/m³ (default: `8500.0`).
- `--pivot_mass <float>`: Explicit pivot mass override in kg (default: `None`).
- `--damping <float>`: Viscous rotational damping coefficient b in N·m·s/rad (default: `5.0e-08`).
- `--damping_noise <float>`: Relative uniform random per-needle damping variation (default: `0.0`).

1.8.3 Time integration and avalanche detection

- `--t_sim <float>`: Total simulated time in seconds; ignored for `forc`/`demag_rot`/`demag_linear` modes, which compute their own duration. In `hysteresis` mode it cannot be combined with `--sweep_rate_dimless` (default: `2.0`, taken from `config.yaml`).
- `--dt_factor <float>`: Integration step as a fraction of the worst-case local period $T_{\mathrm{loc}} = 2\pi/\omega_{\mathrm{loc}}$, where $\omega_{\mathrm{loc}} = \sqrt{mB_{\mathrm{loc}}/I}$ and B_{loc} bounds the field any needle can feel in any configuration (dipolar bound $\max_i \max_{\hat{u}} \sum_j |A_{ij}\hat{u}|\, m$ plus the maximum applied field). This guarantees $\omega_{\mathrm{loc}}\Delta t \leq 2\pi$ `dt_factor` (default: `0.025`, i.e. ≤ 0.157). Before October 2026, Δt was a fraction of T_0 (built from B_{ref}), which left $\omega_{\mathrm{loc}}\Delta t \approx 0.5$. T_0 is still used as the time unit for dwell times and sweep rates.
- `--noise <float>`: Initial angular noise amplitude in radians (default: `1.5`).
- `--seed <int>`: RNG seed; time-derived if omitted (default: `None`).
- `--log_every <int>`: Write one CSV row every N integration steps (default: `10`).
- `--log_adaptive <float>`: If `> 0`, also write a row on any step where `|delta M_proj|` since the last logged row exceeds this threshold, or a flip event just committed; `0.0` disables this and reproduces exactly the `--log_every` cadence (default: `0.0`).
- `--flip_angle_deg <float>`: Rest-angle displacement threshold for the `flip_angle` channel (default: `90.0`).
- `--flip_band_deg <float>`: Schmitt dead-band half-width around the perpendicular to the drive axis, for the `flip_field` channel (default: `30.0`).
- `--flip_dwell_T0 <float>`: Dwell time required to commit a flip, in units of T_0 (default: `0.5`).
- `--flip_settle_frac <float>`: `|omega|` settling threshold as a fraction of ω_0, for the `flip_angle` channel (default: `0.05`).
- `--event_log`: Write a per-committed-event CSV (`step,t,needle_id,channel,theta`) for offline avalanche clustering (default: off).
- `--dt_guard_alpha <float>`: Stability-monitor threshold on `max_i sqrt(m|B_i|/I)*dt` (default: `0.2`; never reached with the default `dt_factor`).
- `--dt_guard_substep`: Re-integrate flagged steps with 4 global sub-steps; breaks strict symplecticity, exploratory (default: off).

1.8.4 Dipolar cutoff and boundaries

- `--cutoff_shells <float>`: Interaction cutoff in multiples of r_{nn}; `<= 0` means no cutoff, i.e. the full pair sum (default: `0.0`). The tensor is dense, so a cutoff gives no speed-up; it only biases the field (a 3.5 r_{nn} cutoff underestimates the interior field of an ordered array by ~15–17 %). Use a finite value only for convergence studies.
- `--cutoff_m <float>`: Absolute cutoff in meters; overrides `--cutoff_shells` when `> 0` (default: `None`).
- `--pbc`: Enable periodic boundary conditions via finite image sums (default: off).
- `--n_images <int>`: Number of periodic images per direction (default: `1`).
- `--tensor_mem_limit_gb <float>`: Memory guard on the $K \times K$ dipolar tensor (default: `6.0`).
- `--float32`: Use float32 tensor/state instead of float64 (default: off; double precision preferred for final results).

1.8.5 External field

- `--field_mode {static,hysteresis,hysteresis_qs,forc,sine,step_up,step_pos,step_down,step_neg,pulse,de` (default: `static`).
- `--qs_dB_factor <float>`: Field step of `hysteresis_qs`, in units of B_{ref} (default: `0.005`).
- `--qs_tol_torque <float>`, `--qs_tol_omega <float>`: Rest criteria of `hysteresis_qs`, as fractions of mB_{lat} and ω_{lat} (default: `1e-5` each).
- `--qs_check_every <int>`: Integration steps between rest checks (default: `5`).
- `--qs_max_relax_tau <float>`: Safety cap on the relaxation time per field step, in units of τ_{relax} (default: `2000`).
- `--qs_dynamics {inertial,overdamped}`: Relaxation dynamics at fixed field (default: `inertial`).
- `--qs_kick <float>`: Saddle check: random kick in rad applied after the first rest, followed by a second relaxation (default: `1e-4`; `0` disables).
- `--qs_state_log`: Save the rest-state angles of every field step to `states/<tag>_qs_states.npz`.
- `--qs_stop_after <int>`: Stop after this many field steps (partial loop, for targeted reruns; default `0` = full loop).
- `--qs_od_cfl <float>`: Euler step of the overdamped mode, as a fraction of $b/(mB_{\mathrm{loc}})$ (default: `0.2`).

- `--B_ext <float>`: Field amplitude in Tesla; if omitted, `B_max_factor * B_ref` is used (default: `None`).
- `--B_max_factor <float>`: `B_ext = factor * B_ref` when `--B_ext` is omitted (default: `8.0`).
- `--phi_ext_deg <float>`: Field direction in degrees relative to $+x$ (default: `0.0`).
- `--field_freq <float>`: Sine-mode frequency in Hz (default: `1.0`).
- `--field_delay <float>`: Delay before step/pulse protocols activate, in seconds (default: `0.0`).
- `--t_pulse <float>`: Pulse duration in seconds (default: `None`, i.e. field stays on indefinitely after the delay).
- `--hyst_spacing {linear,log}`: Time-warping of the hysteresis ramp within each segment (default: `linear`).
- `--hyst_log_k <float>`: Log-spacing concentration parameter (default: `5.0`).
- `--hyst_slow_window <B_lo,B_hi>`: Divides the sweep rate by `--hyst_slow_factor` while `|B|` is inside this window, on both up- and down-sweeps (default: `None`, disabled).
- `--hyst_slow_factor <float>`: Sweep-rate divisor inside `--hyst_slow_window`; `1.0` is a no-op (default: `1.0`).
- `--sweep_rate_dimless <float>`: Dimensionless sweep rate $r = (dB/dt)\,\tau_{\mathrm{relax}}/B_{\mathrm{ref}}$ (see Section 1). Sets the ramp of `hysteresis` (instead of `--t_sim`) and `forc` (instead of `--forc_rate`) runs (default: `None`, i.e. the ramp comes from `--t_sim` / FORC flags as before).

1.8.6 FORC protocol

- `--forc_Br_min <float>`: Minimum reversal field in Tesla; defaults to $-B_{\mathrm{max}}$ (default: `None`).
- `--forc_n_curves <int>`: Number of minor reversal curves (default: `30`).
- `--forc_t_sat <float>`: Hold time at $+B_{\mathrm{max}}$ before each curve, in seconds (default: `0.05`).
- `--forc_t_ramp_down <float>`: Ramp-down time to B_r, used only if `--forc_rate` is unset (default: `0.10`).
- `--forc_t_ramp_up <float>`: Ramp-up time back to $+B_{\mathrm{max}}$, used only if `--forc_rate` is unset (default: `0.20`).
- `--forc_rate <float>`: Constant dB/dt in T/s; overrides the fixed ramp times above, computing each curve's duration to enforce this rate (default: `None`).

1.8.7 Demagnetization

- `--demag_freq <float>`: Rotation/oscillation frequency of the demag field in Hz (default: `2.0`).
- `--demag_cycles <int>`: Number of decay cycles (default: `20`).
- `--t_relax_after <float>`: Extra zero-field relaxation time appended after demag completes, in seconds (default: `2.0`).

1.8.8 Output and performance

- `--out_dir <path>`: Output root directory (default: `compassV2_output`).
- `--tag <str>`: Run tag; auto-built from geometry/mode/N/M/seed if omitted (default: `None`).
- `--use_gpu`: Use the CuPy backend (default: off).
- `--progress`: Print a progress bar during integration (default: off).
- `--verbose`: Print a derived-quantity summary before running (default: off).
- `--make_plot`: Generate an additional CSV diagnostic quicklook PNG; lattice PNGs are generated automatically regardless (default: off).
- `--png_dpi <int>`: Resolution for the automatic lattice PNGs (default: `300`).
- `--png_transparent`: Transparent background for lattice PNGs (default: off).
- `--png_with_axes`: Include axes/grid/title on lattice PNGs (default: off).
- `--png_no_panel_titles`: Remove "Initial state"/"Final state" panel titles from the side-by-side PNG (default: off).
- `--domain_tol_deg <float>`: Angular tolerance in degrees for the final connected-component domain-statistics calculation (default: `15.0`).

Flags with no `help=` string in the source (defaults verified directly from `build_parser()`, not from prose): `--needle_thickness`, `--steel_density`, `--pivot_radius`, `--pivot_thickness`, `--pivot_density`, `--pivot_mass`, `--seed`, `--tensor_mem_limit_gb`, `--forc_n_curves`, `--forc_t_sat`, `--forc_t_ramp_down`, `--forc_t_ramp_up`, `--demag_freq`, `--demag_cycles`, `--out_dir`, `--tag`, `--use_gpu`, `--progress`, `--verbose`, `--domain_tol_deg`.

There is **no** `--video`, `--frame_every`, `--fps`, `--dpi`, `--make_images`, `--halo_mode`, `--csv_order`, `--torque_tol`, `--t_sim_full`, `--demag_temp`, `--demag_delay`, `--forc_sweep`, `--pbc_images`, `--gpu`, `--ext_Bx`/`--ext_By`, `--phi_ext`, `--domain_tol`, or `--progress_bar` flag in the current engine — these all belonged to an earlier, now-

archived lineage (`archive/compass.py.bkp/.old`). If you have old commands or notes using any of these, translate them via this guide before running them.

1.9 7. Output Files & Data Formats

Each run writes a tagged directory tree under `--out_dir`, with four subdirectories:

- `data/<tag>.csv` — the time-series CSV.
- `meta/<tag>.json` — JSON metadata (config, derived quantities, provenance).
- `states/<tag>_initial.npz` and `states/<tag>_final.npz` — compressed state snapshots (`xs`, `ys`, `theta`, `omega`, `r_nn`, and an embedded `metadata_json` field).
- `images/<tag>_initial_lattice.png`, `images/<tag>_final_lattice.png`, `images/<tag>_initial_final.png` — automatic lattice figures, generated for **every** run regardless of `--make_plot`.

If `--make_plot` is set, an additional `<tag>_quicklook.png` diagnostic time-series plot is written directly under `--out_dir`. If `--event_log` is set, a per-committed-event CSV is written to `data/<tag>_events.csv`.

1.9.1 CSV columns (`data/<tag>.csv`)

```
step,t_s,Bx_T,By_T,B_scalar_T,branch,forc_index,Mx,My,M_proj,S1,S2,
theta_director_rad,q_axis,flip_field,flip_angle,E_dip_J,E_ext_J,E_kin_J,
E_total_J,omega_rms_rad_s,omega_max_rad_s,W_field_J,E_diss_J
```

`branch` is the field-protocol's current segment label (e.g. `0_to_pos`, `pos_to_0` for hysteresis; `sat`/`down`/`up` for FORC). `forc_index` identifies the active FORC curve, `-1` otherwise. `flip_field` and `flip_angle` are **committed** avalanche-event counts accumulated since the previous logged row, not instantaneous states.

`W_field_J` and `E_diss_J` form the **energy ledger**. They are cumulative from $t = 0$ and use trapezoidal integration consistent with the integrator:

- $W_{\text{field}}(t) = -\sum_i \int \mathbf{m}_i \cdot d\mathbf{B}_{\text{ext}}$ is the work of the time-dependent applied field;
- $E_{\text{diss}}(t) = \int \sum_i b_i \, \omega_i^2 \, dt$ is the viscous dissipation.

They satisfy $E_{\text{total}}(t) - E_{\text{total}}(0) = W_{\text{field}}(t) - E_{\text{diss}}(t)$ up to $O(\Delta t^2)$, with a larger error at field jumps (step/pulse). Over a closed hysteresis cycle, W_{field} equals the loop area $-Km \oint M_{\text{proj}} \, dB$. The metadata summarizes the balance in `energy_ledger` (`balance_residual_rel`), which is a per-run quality-control number.

1.9.2 Metadata JSON (`meta/<tag>.json`)

Records `program` (`"compass.py"`), `version`, `source_file_timestamp` (derived from the file's own mtime), creation timestamps, the full effective configuration, and derived quantities (r_{nn}, B_{ref}, B_{eff}, ω_0, T_0, T_{loc}, Δt, Q ($= Q_0$), B_{lat}, `Q_ref`, `Q_field`, cutoff policy, the reported sweep rate, the `energy_ledger` summary, the `sweep_rate` block (for hysteresis/FORC runs: r, dB/dt, τ_{relax} and its regime, and which input set the rate), and — for hysteresis runs — the full explicit per-segment schedule if `--hyst_slow_window` was used).

1.9.3 Regenerating images later

`compass_generate_images.py --run_dir <out_dir> [--recursive]` regenerates lattice PNGs from already-saved CSV/NPZ output (e.g. after changing `--dpi`/`--transparent`/style), without re-running the simulation.

1.10 8. Step-by-Step Practical Examples

1.10.1 Example 1: Basic smoke test (no field)

Run a small 8×8 square lattice from a randomized state and let it relax under dipolar interaction alone.

```
python3 compass.py --geometry square --N 8 --M 8 --t_sim 2.0 --noise 2.5 \
    --out_dir ~/results/smoke --tag smoke_relax --verbose
```

1.10.2 Example 2: Hysteresis loop

Trace a complete magnetization cycle for a 10×10 triangular lattice with a 1.5 mT applied field at a 30° offset.

```
python3 compass.py --geometry triangular --N 10 --M 10 \
    --field_mode hysteresis --B_ext 1.5e-3 --phi_ext_deg 30 \
    --t_sim 5.0 --damping 5e-7 \
    --out_dir ~/results/hyst_demo --tag triangular_hyst_demo
```

1.10.3 Example 3: Hysteresis with a slow-sweep window

Same as above, but sweep four times more slowly while $|B|$ is between 0.5 and 1.0 mT (on both branches), and log every step whenever M_{proj} jumps by more than 0.01 between samples.

```
python3 compass.py --geometry square --N 20 --M 20 \
    --field_mode hysteresis --B_ext 1.5e-3 \
    --hyst_slow_window 0.0005,0.001 --hyst_slow_factor 4.0 \
    --log_adaptive 0.01 \
    --out_dir ~/results/hyst_slow --tag square_hyst_slow
```

1.10.4 Example 3b: Hysteresis at a controlled dimensionless rate

Sweep at $r = 10^{-3}$ (close to quasi-static for small arrays); the ramp duration follows from r, $B_{\max}$ and τ_{relax}.

```
python3 compass.py --geometry triangular --N 6 --M 6 \
    --field_mode hysteresis --B_max_factor 3 --damping 1e-7 \
    --sweep_rate_dimless 1e-3 --verbose \
    --out_dir ~/results/hyst_rate --tag tri_r1e-3
```

1.10.5 Example 4: Demagnetization then a field pulse

Rotationally demagnetize at 5 Hz over 40 cycles, relax for 0.5 s, in a separate run apply a 100 μT pulse at 90° for 0.5 s.

```
python3 compass.py --field_mode demag_rot --demag_freq 5.0 --demag_cycles 40 \
    --t_relax_after 0.5 --out_dir ~/results/demag --tag demag_rot_demo

python3 compass.py --field_mode pulse --B_ext 100e-6 --phi_ext_deg 90 \
    --field_delay 0.2 --t_pulse 0.5 --t_sim 1.5 \
    --out_dir ~/results/pulse --tag pulse_demo
```

1.10.6 Example 5: Large-scale run on GPU

Run a 30×30 square lattice (900 needles) using the CuPy backend.

```
python3 compass.py --geometry square --N 30 --M 30 --use_gpu \
    --field_mode hysteresis --t_sim 5.0 \
    --out_dir ~/results/gpu_run --tag square_30x30_gpu
```

1.11 9. Running Sweep Campaigns

For parametric studies (e.g. how damping/Q alters hysteresis loops and avalanche statistics across geometries), use **`damping_sweepV03.py`** — the only campaign wrapper currently compatible with `compass.py`'s API (`damping_sweep.py`, `damping_sweepV02.py`, and `sweep_damping_hysteresis.py` all call functions removed from the engine in July 2026 and cannot run at all; see `docs/AUDIT.md` bugs B1/B3 — do not use them).

```
python3 damping_sweepV03.py --out_dir ./damping_campaign --grid_n 20 \
    --n_seeds 3 --n_dampings 5 --geometries square,triangular
```

To make the sweep rate a campaign axis ($Q_0 \times r \times$ geometry $\times$ seed), pass `--sweep_rates_dimless "0.1,0.01,0.001"`. The hysteresis and FORC stages are then repeated for each r (tags get a suffix `_rNN`, and `manifest.csv` gets the columns `r_idx` and `sweep_rate_dimless`); `--t_sim_periods` and the FORC ramp flags are ignored for those stages, and the relax stage runs once. Extract the quasi-static $A_0(Q_0) = \lim_{r \to 0} A(r, Q_0)$

from these runs. Note that `damping_sweep_analysis.py` still groups by `(geometry, damp_idx)` only and does not yet separate rates.

Notable flags: `--n_workers N` runs N campaign entries in parallel (separate processes); `--resume 1` skips runs already present in `manifest.csv` on an interrupted campaign; `--quick_test 1` overrides several parameters to a fast pipeline-validation configuration (`n_seeds=1, n_dampings=3, t_sim_periods=8, grid_n=12`). Run `python3 damping_sweepV03.py --help` for the full flag list (it has its own independent argparse surface, separate from `compass.py`'s).

1.12 10. Summary Reference Table of Parameters

Parameter	Type / Options	Default	Brief Descriptio
Lattice Geometry			
`--geometry`	`square`, `triangular`, `honeycomb`	`square`	Lattice topology
`--N`	`int`	`16`	Rows (or nomi
`--M`	`int`	`16`	Columns (or no
`--R`	`float`	`0.0065`	Half the pivot-t
`--needle_frac`	`float`	`0.80`	Legacy blade-le
`--needle_len`	`float`	`0.010`	Physical blade l
`--needle_width`	`float`	`0.003`	Physical blade v
`--use_legacy_size_from_R`	`0`, `1`	`0`	Derive blade siz
Needle Physical Properties			
`--moment`	`float`	`None`	Override magn
`--inertia`	`float`	`None`	Override mome
`--needle_thickness`	`float`	`0.0004`	Blade thickness
`--steel_density`	`float`	`7850.0`	Blade material
`--steel_Ms`	`float`	`1.59e6`	Saturation mag
`--steel_Bsat`	`float`	`None`	Saturation flux
`--pivot_radius`	`float`	`0.0`	Pivot hole/cylin
`--pivot_thickness`	`float`	`0.0`	Pivot cylinder h
`--pivot_density`	`float`	`8500.0`	Pivot material d
`--pivot_mass`	`float`	`None`	Explicit pivot m
`--damping`	`float`	`5.0e-08`	Viscous dampin
`--damping_noise`	`float`	`0.0`	Relative per-ne
Time Integration & Avalanche Detection			
`--t_sim`	`float`	`None` → `2.0` (config.yaml)	Total simulated
`--dt_factor`	`float`	`0.025`	$\Delta t/T_{\text{loc}}$ (worst-c
`--noise`	`float`	`1.5`	Initial angular
`--seed`	`int`	`None`	RNG seed; time
`--log_every`	`int`	`10`	CSV row caden
`--log_adaptive`	`float`	`0.0`	Extra log rows
`--flip_angle_deg`	`float`	`90.0`	Rest-angle thre
`--flip_band_deg`	`float`	`30.0`	Schmitt dead-b
`--flip_dwell_T0`	`float`	`0.5`	Dwell time to c
`--flip_settle_frac`	`float`	`0.05`	‘
`--event_log`	flag	off	Write a per-eve
`--dt_guard_alpha`	`float`	`0.2`	Stability-monit
`--dt_guard_substep`	flag	off	Sub-step flagge
Dipolar Cutoff & Boundaries			
`--cutoff_shells`	`float`	`0.0`	Cutoff in multi
`--cutoff_m`	`float`	`None`	Absolute cutoff
`--pbc`	flag	off	Periodic bound
`--n_images`	`int`	`1`	Periodic image
`--tensor_mem_limit_gb`	`float`	`6.0`	Memory guard
`--float32`	flag	off	Use float32 ten
External Field			
`--field_mode`	see §3	`static`	Field-vs-time p

Parameter	Type / Options	Default	Brief Descriptio
`--B_ext`	`float`	`None`	Field amplitude
`--B_max_factor`	`float`	`8.0`	B_{ext}
`--phi_ext_deg`	`float`	`0.0`	Field direction
`--field_freq`	`float`	`1.0`	Sine-mode freq
`--field_delay`	`float`	`0.0`	Delay before st
`--t_pulse`	`float`	`None`	Pulse duration
`--hyst_spacing`	`linear`, `log`	`linear`	Time-warping
`--hyst_log_k`	`float`	`5.0`	Log-spacing co
`--hyst_slow_window`	`"B_lo,B_hi"`	`None`	Slow-sweep wi
`--hyst_slow_factor`	`float`	`1.0`	Sweep-rate divi
`--qs_dB_factor`	`float`	`0.005`	Field step of hy
`--qs_tol_torque`	`float`	`1e-5`	Rest criterion o
`--qs_tol_omega`	`float`	`1e-5`	Rest criterion o
`--qs_check_every`	`int`	`5`	Steps between
`--qs_max_relax_tau`	`float`	`2000`	Cap on relaxati
`--qs_dynamics`	`inertial`, `overdamped`	`inertial`	Relaxation dyn
`--qs_kick`	`float`	`1e-4`	Saddle-check k
`--qs_state_log`	flag	off	Save rest-state a
`--qs_stop_after`	`int`	`0`	Stop after N fie
`--qs_od_cfl`	`float`	`0.2`	Euler step of th
FORC Protocol			
`--forc_Br_min`	`float`	`None`	Minimum reve
`--forc_n_curves`	`int`	`30`	Number of min
`--forc_t_sat`	`float`	`0.05`	Hold time at +
`--forc_t_ramp_down`	`float`	`0.10`	Ramp-down tir
`--forc_t_ramp_up`	`float`	`0.20`	Ramp-up time
`--forc_rate`	`float`	`None`	Constant dB/d
`--sweep_rate_dimless`	`float`	`None`	Dimensionless
Demagnetization			
`--demag_freq`	`float`	`2.0`	Demag field fre
`--demag_cycles`	`int`	`20`	Number of dec
`--t_relax_after`	`float`	`2.0`	Extra relaxatio
Output & Performance			
`--out_dir`	`str`	`compassV2_output`	Output root dir
`--tag`	`str`	`None`	Run tag; auto-b
`--use_gpu`	flag	off	Use the CuPy b
`--progress`	flag	off	Print a progress
`--verbose`	flag	off	Print a derived-
`--make_plot`	flag	off	Extra CSV diag
`--png_dpi`	`int`	`300`	Lattice PNG res
`--png_transparent`	flag	off	Transparent lat
`--png_with_axes`	flag	off	Show axes/grid
`--png_no_panel_titles`	flag	off	Remove panel t
`--domain_tol_deg`	`float`	`15.0`	Angular toleran

1.13 11. Validation & Quality Control

1.13.1 Automated tests

```
python3 -m pytest -q tests
```

- `tests/test_physics.py` checks the physics against independent calculations. It covers the tensor against the direct dipole-dipole sum, torque $= -\partial E/\partial\theta$, second-order energy conservation at $b = 0$, energy decay with damping, the full-sum default, the dt bound, protocol independence of Q_0, the energy balance $\Delta E_{\text{total}} = W_{\text{field}} - E_{\text{diss}}$, loop area = field work, and convergence of the loop area as $r \to 0$.
- `tests/test_reference_runs.py` checks that the numbers did not change (frozen 3×3 runs). When a physics default changes on purpose, regenerate the references with `python3 tests/generate_reference.py` in

the same commit.
- Known physics defects are encoded as `xfail(strict=True)` tests until they are fixed (none are open as of October 2026).

1.13.2 Per-run checks (metadata JSON)

Field	Healthy value
`stability_monitor.flagged_steps`	`0`
`stability_monitor.max_omega_local_dt`	≤ 0.16 with the default `dt_factor`
`energy_ledger.balance_residual_rel`	a few $\times 10^{-3}$ or less (the manual's examples give 2×10^{-4}–2×10^{-3})
`derived.cutoff_policy`	`full_sum`
`sweep_rate.sweep_rate_dimless` (hysteresis/FORC)	r small enough that the loop area has converged

1.13.3 Before quoting a physical result

1. **Rate:** show convergence in r (quasi-static limit) or report the result explicitly as a function of r.
2. **Time step:** repeat representative runs at `dt_factor` halved.
3. **Size and boundaries:** repeat at ≥ 3 array sizes. With free boundaries, edge fields are part of the physics of a finite array.
4. **Seeds:** aggregate over seeds per (geometry, Q_0, r) condition before fitting anything.
5. **Avalanches:** compare the `flip_field`, `flip_angle` and `--event_log` channels. Test a power law against log-normal and stretched-exponential alternatives, and report when the tail is too short.